Course Mini-Project: Binary Image Denoising via the Ising Model & Markov Random Fields

Department of Applied Mathematics and Statistics

Academic Semester 2:May, 2026

1 Project Overview

In this mini-project, you will implement a classic computer vision application of statistical mechanics: **Bayesian Image Denoising** using the 2D **Ising Model**. By treating a digital image as a grid of interacting magnetic spins (a Markov Random Field), you will formulate image recovery as an energy minimization problem. You will implement two foundational optimization algorithms from scratch using Python and NumPy: a deterministic greedy approach called **Iterated Conditional Modes (ICM)** and a stochastic physics-inspired approach called **Simulated Annealing (SA)**.

1.1 Learning Objectives

By completing this project, you will demonstrate the ability to:

- Bridge statistical mechanics (Ising model) and probabilistic graphical models (Markov Random Fields).
- Map pixel values, structural smoothness, and noisy observations to an energy function.
- Implement pixel-level local optimization algorithms without high-level external libraries.
- Perform hyperparameter tuning and quantitatively evaluate results using pixel accuracy.

2 Theoretical Background

2.1 The 2D Ising Model

The Ising model was originally formulated in physics to explain ferromagnetism. It assumes a discrete lattice of sites, where each site i contains a spin $\sigma_i \in \{-1, +1\}$, representing magnetic moments pointing "up" $(+1)$ or "down" (-1). Spins interact with their nearest neighbors to minimize the system's total energy, defined by the Hamiltonian:

$$H(\sigma) = -J \sum_{\langle i,j \rangle} \sigma_i \sigma_j - h \sum_i \sigma_i \tag{1}$$

where $\langle i, j \rangle$ denotes a summation over all adjacent pairs (nearest neighbors), J is the coupling constant ($J > 0$ favors alignment, representing a ferromagnet), and h is an external magnetic field forcing spins to align with a specific direction.

2.2 Markov Random Fields and Image Denoising

In computer vision, we can map this framework directly to a binary image. Let Y be an observed, corrupted binary image, where each pixel $y_i \in \{-1, +1\}$. Let X be the unobserved,

clean binary image we wish to reconstruct, where $x_i \in \{-1, +1\}$ represents the true hidden state of pixel i.

We model the joint distribution using a Markov Random Field (MRF). The energy configuration of the hidden image X given the observed data Y is formulated as:

$$E(X|Y) = -J \sum_{\langle i,j \rangle} x_i x_j - h \sum_i x_i y_i \tag{2}$$

- **The Smoothness Term ($-J \sum x_i x_j$):** Penalizes spatial discontinuity. If $J > 0$, neighboring pixels are encouraged to share the same value (mitigating random noise).
- **The Data Fidelity Term ($-h \sum x_i y_i$):** Penalizes deviations from the observed noisy image. If $h > 0$, the hidden pixel x_i is encouraged to match the observed value y_i.

2.3 Local Energy Changes (ΔE)

Computing the global energy $E(X|Y)$ across the entire image at every step of an optimization loop is computationally intractable ($O(N^2)$ or worse). Because the MRF satisfies the local Markov property, the change in energy ΔE_i resulting from flipping a single hidden pixel $x_i \to -x_i$ depends only on its immediate 4-connected neighbors ($N(i)$):

$$\Delta E_i = E(x_i^{\text{new}}) - E(x_i^{\text{old}}) = 2x_i^{\text{old}} \left(J \sum_{j \in N(i)} x_j + h y_i \right) \tag{3}$$

You will use this localized equation to drive your optimization loops efficiently.

3 Project Tasks

3.1 Task 1: Synthetic Data Generation and Noise Injection

Implement a function to generate a clean 64×64 binary image containing clear geometric structures (e.g., nested squares or a frame). Map the values explicitly to $\{-1, +1\}$ (where -1 represents background black pixels and $+1$ represents foreground white pixels). Implement an independent flip-noise generator that flips each pixel with a probability $p = 0.15$ to produce the noisy image Y.

3.2 Task 2: Local Energy Difference Engine

Implement the function `calculate_local_energy_change(X, Y, r, c, J, h)`. Given the current hidden matrix X, observed matrix Y, pixel coordinate (r, c), and parameters J and h, return the exact scalar ΔE if $X[r, c]$ were to be multiplied by -1. Ensure you handle boundary conditions correctly (pixels along edges or corners have fewer than 4 neighbors).

3.3 Task 3: Deterministic Optimization via ICM

Implement the Iterated Conditional Modes (ICM) algorithm. ICM is a coordinate descent algorithm that greedily minimizes energy.

1. Initialize $X = Y$.
2. For each pixel (r, c) in the grid, calculate ΔE. If $\Delta E < 0$, accept the flip ($X[r, c] \leftarrow -X[r, c]$).
3. Repeat full passes over the image until convergence (no pixels change state during a full pass) or until reaching a maximum number of iterations.

3.4 Task 4: Stochastic Optimization via Simulated Annealing

Implement the Simulated Annealing algorithm to escape local minima using thermal fluctuations.

1. Initialize $X = Y$ and set an initial high temperature $T_{\text{init}} = 5.0$.
2. Uniformly select a random pixel (r, c), and compute ΔE.
3. If $\Delta E < 0$, accept the flip. If $\Delta E \geq 0$, accept the flip with Boltzmann probability $P = \exp(-\Delta E/T)$.
4. Cool down the system using a geometric schedule $T \leftarrow \alpha T$ (e.g., $\alpha = 0.95$) after every full grid-sweep equivalent (64×64 steps).

3.5 Task 5: Quantitative Evaluation & Analysis

Vary the ratio J/h across a designated search space (e.g., $J \in [0.0, 2.0]$, $h = 1.0$) and plot the resulting Pixel Reconstruction Accuracy against the hyperparameters. Document and visually contrast what occurs when $J \gg h$ versus $J \ll h$.

4 Python Solution Template

Use the following structured skeleton to complete your project. Do not modify the function signatures.

```
import numpy as np
import matplotlib.pyplot as plt

def generate_synthetic_image(size=64):
img = -1 * np.ones((size, size), dtype=np.int32)
img[15:45, 15:45] = 1
img[22:38, 22:38] = -1
return img

def add_noise(clean_img, noise_prob=0.15):
noisy_img = clean_img.copy()
flip_mask = np.random.rand(*clean_img.shape) < noise_prob
noisy_img[flip_mask] *= -1
return noisy_img

def calculate_local_energy_change(x, y, r, c, J, h):
current_val = x[r, c]
observed_val = y[r, c]
rows, cols = x.shape

neighbors = []
if r > 0:        neighbors.append(x[r-1, c])
if r < rows - 1: neighbors.append(x[r+1, c])
if c > 0:        neighbors.append(x[r, c-1])
if c < cols - 1: neighbors.append(x[r, c+1])

# ------------------------------------------------------------
# TODO: Calculate Delta E based on neighbors and observed_val
# ------------------------------------------------------------
return 0
def denoise_icm(noisy_img, J=1.0, h=0.7, max_iters=10):
X = noisy_img.copy()
rows, cols = X.shape
```

```
for iteration in range(max_iters):
changes = 0
for r in range(rows):
for c in range(cols):
delta_E = calculate_local_energy_change(X, noisy_img, r, c, J, h)
# ------------------------------------------------------------
# TODO: Implement local greedy acceptance criteria
# ------------------------------------------------------------
pass
if changes == 0:
break
return X
```

5 Grading Rubric

Your submission will be graded according to the guidelines presented in Table 1.

Table 1: Project Grading Criteria

Component	**Weight**	**Assessment Criteria**
Mathematical Correctness	25%	Verification of local energy derivation (ΔE) and boundary chec
ICM Algorithm Implementation	25%	Correct greedy update framework and clean convergence termi
Simulated Annealing	20%	Proper Metropolis step execution and correct cooling updates.
Hyperparameter Analysis	20%	Detailed diagnostic report and visualization of the J/h parame
Code Structure & Clarity	10%	Adherence to functional requirements, vectorized practices, and